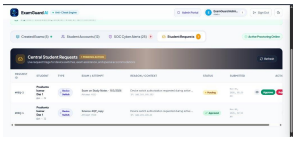
EXAMGUARD AI: DESIGN AND DEVELOPMENT OF AN ONLINE EXAMINATION SYSTEM WITH AI-ASSISTED PROCTORING AND EXAMINATION INTEGRITY MONITORING

A Multi-Sensor Browser-Gated Proctoring Platform and Cloud Assessment Engine

A PROJECT REPORT
Submitted by

PRASHANTA KUMAR DAS
(Regd. No: 240301120547)

in partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING



SCHOOL OF ENGINEERING AND TECHNOLOGY,
BHUBANESWAR CAMPUS
CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT ODISHA

July 2026 - November 2026

BONAFIDE CERTIFICATE

This is to certify that this project report entitled "EXAMGUARD AI: DESIGN AND DEVELOPMENT OF AN ONLINE EXAMINATION SYSTEM WITH AI-ASSISTED PROCTORING AND EXAMINATION INTEGRITY MONITORING" is the bonafide work of PRASHANTA KUMAR DAS (240301120547) who carried out the project work under my supervision in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering. To the best of my knowledge, this project has not been submitted previously for the award of any other degree or diploma in this institute or any other university.

SIGNATURE
Mr. RAJESH GHOSH
Project Guide [Official Designation: _____]
Department of Computer Science & Engineering
SoET, Bhubaneswar Campus
Centurion University of Technology and Management

Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university

SIGNATURE
[Head of Department Name: _____]
HEAD OF THE DEPARTMENT
Department of Computer Science & Engineering
SoET, Bhubaneswar Campus
Centurion University of Technology and Management

DEPARTMENT SEAL

DECLARATION

I hereby declare that the project entitled "EXAMGUARD AI: DESIGN AND DEVELOPMENT OF AN ONLINE EXAMINATION SYSTEM WITH AI-ASSISTED PROCTORING AND EXAMINATION INTEGRITY MONITORING", submitted for the Skill Project / Mini-Project requirement of the 5th semester Bachelor of Technology in Computer Science and Engineering, is an authentic record of my original work conducted at the School of Engineering and Technology, Centurion University of Technology and Management, Bhubaneswar Campus.

I further confirm that this work has not formed the basis for the award of any other degree, diploma, fellowship, or other similar title at this or any other university or institution.

Name and Signature of the Student:

PRASHANTA KUMAR DAS (240301120547)

Place: Jatni, Bhubaneswar, Odisha
Date: / 11 / 2026

ACKNOWLEDGEMENTS

I express my deep sense of gratitude and indebtedness to my faculty supervisor, Mr. Rajesh Ghosh, for his invaluable guidance, constructive critiques, and constant encouragement throughout the lifecycle of this project. His deep insights into web architectures, client-side computer vision models, and application security enabled me to approach both theoretical foundations and full-stack software implementation with clarity and precision.

I also extend my sincere gratitude to the academic leadership and administration of the School of Engineering and Technology, Bhubaneswar Campus, for their administrative coordination, laboratory facilities, and encouragement during the academic term.

I convey my appreciation to the Head of the Department of Computer Science and Engineering, for providing the software tools and departmental support necessary to complete this project smoothly.

My heartfelt thanks are extended to the technical staff of the Computer Science laboratories for their prompt assistance. Lastly, I thank my family members and peers for their boundless patience, motivation, and mutual cooperation that made this engineering endeavor an enriching learning experience.

Name of the Student:
PRASHANTA KUMAR DAS
(240301120547)

Place: Jatni, Bhubaneswar, Odisha
Date: / 11 / 2026

ABSTRACT

The transition toward computer-based assessments in higher education has accentuated the conflict between institutional examination integrity and the operational overhead of physical invigilation. Conventional remote proctoring suites frequently rely on proprietary kernel-level monitoring binaries or invasive browser extensions that induce severe privacy concerns, heavy CPU utilization, and high licensing costs. To overcome these barriers, this project presents the complete design, architectural implementation, and verification of ExamGuard AI, an enterprise-grade, browser-native proctored examination platform engineered in Python Flask and client-side machine learning technologies.

ExamGuard AI incorporates multi-sensor vision proctoring executed client-side within the candidate's browser using TensorFlow.js (BlazeFace 6-point anatomical facial landmark detector and COCO-SSD object classifier). The client-side pipeline implements intelligent head pose estimation based on facial landmark geometry, utilizing an asymmetric horizontal Yaw ratio and vertical Pitch ratio that permits necessary desk scrap-paper scratchpad calculations while flagging unauthorized head turns, multi-face presence, and prohibited peripherals (e.g., cell phones). Real-time examination security is enforced via DOM focus gating, full-screen lockdown telemetry, dynamic device switching triage, and a push-to-talk WebM voice intercom allowing proctors to transmit instant auditory advisories without third-party plugins.

The persistence tier leverages a dual-database abstraction layer supporting local SQLite (WAL mode) and production cloud PostgreSQL 16 with SSL pooling. An automated test suite comprising 62 unit and integration test fixtures validates database integrity, anti-tampering guards, map coordinate projection, and intercom persistence with 100% test pass rate on local fixtures (61 passed, 1 skipped cloud test). The resulting platform delivers defensible assessment auditing while safeguarding candidate device privacy.

TABLE OF CONTENTS

SECTION	TITLE	PAGE
	CERTIFICATE	ii
	DECLARATION	iii
	ACKNOWLEDGEMENTS	iv
	ABSTRACT	v
	LIST OF FIGURES	ix
	LIST OF TABLES	ix
CHAPTER 1	INTRODUCTION	11
	1.1 Background of Digital Examination Systems	11
	1.2 Problem Statement	11
	1.3 Project Motivation and Scope	11
	1.4 Practical Applications in Modern Higher Education	11
	1.5 Organization of the Report	12
CHAPTER 2	THEORETICAL FOUNDATIONS & LITERATURE REVIEW	13
	2.1 Comparative Analysis of Proctoring Paradigms	13
	2.2 Client-Side Machine Learning and Facial Landmark Detection	13
	2.3 Head Pose Estimation & Mathematical Angles (Yaw, Pitch, Roll)	13
	2.4 Ethical Proctoring, Privacy Preservation, and Data Sovereignty	14
	2.5 Summary of Existing Shortcomings	14
CHAPTER 3	OBJECTIVES AND SYSTEM REQUIREMENTS	15
	3.1 Primary Engineering Objectives	15
	3.2 Functional Requirements (FR-01 to FR-10)	15
	3.3 Non-Functional Requirements (NFR-01 to NFR-06)	16
	3.4 Hardware, Software, and Environment Specifications	16
CHAPTER 4	SYSTEM ARCHITECTURE & DATABASE DESIGN	17
	4.1 Multi-Tier Architecture & Technology Stack	17
	4.2 Dual-Database Abstraction (SQLite WAL & PostgreSQL 16)	17

	4.3 Relational Entity-Relationship (ER) Schema	17
	4.4 Data Dictionaries for Relational Entities	17
CHAPTER 5	ALGORITHM IMPLEMENTATION & PROCTORING ENGINE	20
	5.1 Client-Side Vision Pipeline (BlazeFace Landmark Geometry)	20
	5.2 Object Detection & Hardware Accessory Detection (COCO-SSD)	20
	5.3 Multi-Sensor Integrity Logic and Temporal Debouncing	20
	5.4 Push-to-Talk WebM Voice Intercom Architecture	21
	5.5 Interactive Multi-Pin SVG Geographical Map Rendering	21
CHAPTER 6	GRAPHICAL USER INTERFACE & LIVE OPERATIONS	23
	6.1 Tutor Command Center Dashboard	23
	6.2 Centralized Student Requests & Device-Switch Authorization	24
	6.3 Cyber Security Operations Center (SOC) Telemetry	24
	6.4 AI Exam & Question Studio Authoring Interface	25
	6.5 Tutor Created Exams and Course Management View	25
	6.6 Individual Candidate Live Proctoring & Session Audit	26
	6.7 Student Authentication & Device-Switch Workflow	26
	6.8 Student Sign-in Interface	27
	6.9 Administrator Login Interface	27
	6.10 Student Registration Interface	28
CHAPTER 7	EXPERIMENTAL RESULTS, VERIFICATION & DISCUSSION	30
	7.1 Automated Pytest Verification (62 Test Fixtures)	30
	7.2 Verification Methodology & Multi-Sensor Integration Analysis	31
	7.3 Edge Case & Fault Tolerance Validation	31
	7.4 Documented System Limitations & Defensible Constraints	31
CHAPTER 8	CONCLUSION AND LEARNING OUTCOMES	33
	8.1 Summary of Project Deliverables	33
	8.2 Technical and Engineering Insights	33
	8.3 Personal and Professional Learning Outcomes	33
CHAPTER 9	FUTURE SCOPE & EXTENSIONS	34

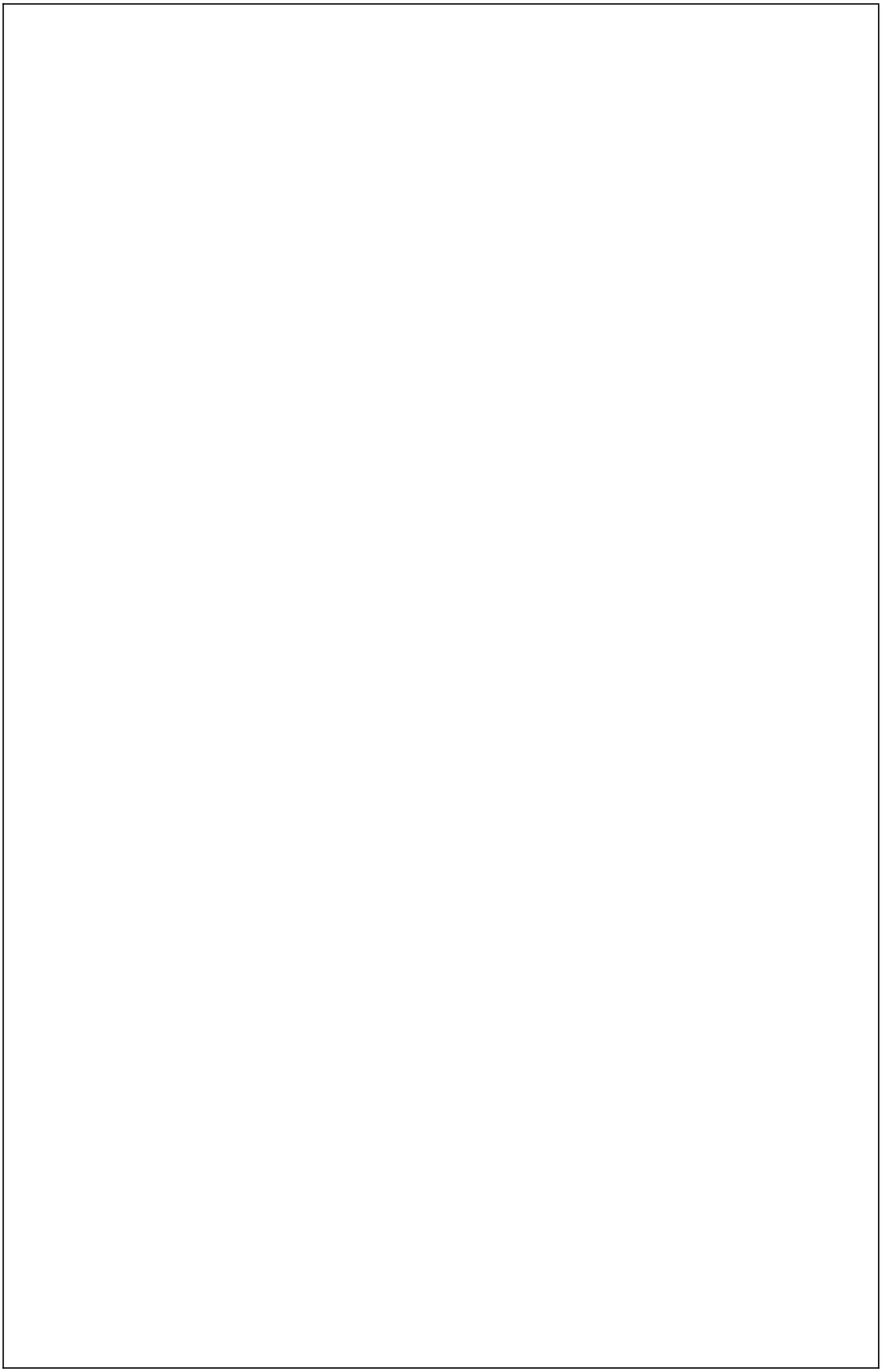
	9.1 WebRTC Peer-to-Peer Bidirectional Audio/Video Streams	34
	9.2 Offline-First IndexedDB Local Exam State Synchronization	34
	9.3 Native Mobile MobileNet Proctoring SDKs	34
CHAPTER 10	REFERENCES & BIBLIOGRAPHY	35
APPENDIX	COURSE OUTCOME ATTAINMENT & SETUP SCRIPTS	36

LIST OF FIGURES

FIGURE	CAPTION / TITLE	PAGE
Fig 4.1	Multi-Tier System Architecture of ExamGuard AI	17
Fig 4.2	Relational Database Entity-Relationship (ER) Diagram	17
Fig 5.1	Client-Side AI Proctoring Pipeline and Pose Estimation Flowchart	20
Fig 5.2	Push-to-Talk Tutor Voice Intercom Sequence Diagram	21
Fig 5.3	Interactive Outline Map of India with 5 Numbered Pins	22
Fig 6.1	Tutor Command Center Dashboard with Real-Time Assessment Metrics	23
Fig 6.2	Centralized Student Requests Management and Device-Switch Triage	24
Fig 6.3	Cyber Security Operations Center (SOC) SIEM Incident Telemetry Log	25
Fig 6.4	AI Exam & Question Studio Authoring Interface	25
Fig 6.5	Tutor Created Exams and Course Management View	25
Fig 6.6	Individual Candidate Live Proctoring & Session Audit Console	26
Fig 6.7	Student Login Portal and Active Device-Switch Security Workflow	26
Fig 6.8	Student Sign-in Interface	27
Fig 6.9	Administrator Login Interface	28
Fig 6.10	Student Registration Interface	28

LIST OF TABLES

TABLE	TITLE	PAGE
Table 2.1	Comparative Analysis of Commercial Proctoring Systems vs. ExamGuard AI	14
Table 3.1	Functional Requirements Specification Matrix	15
Table 3.2	Software and Hardware Operating Environment Specifications	16
Table 4.1	Relational Schema Data Dictionary: Users and Roles	17
Table 4.2	Relational Schema Data Dictionary: Examinations and Submissions	18
Table 4.3	Relational Schema Data Dictionary: Violations, Intercom and Device Requests	18
Table 5.1	Sensor Thresholds, Temporal Debounce Limits, and Action Triggers	20
Table 7.1	Automated Pytest Execution Summary across Core Functional Test Modules	30



CHAPTER 1: INTRODUCTION

1.1 Background of Digital Examination Systems

The transition toward computer-based assessments across higher education institutions and certification bodies has transformed the operational logistics of administering standardized evaluations. By replacing physical paper booklets with dynamic web portals, universities have achieved instantaneous grading of objective assessments, automated result compilation, and scalable candidate distribution across geographic territories. However, this transition has simultaneously exposed academic institutions to unprecedented vulnerabilities in examination security. Unmonitored network environments, unauthorized web browsing, collaboration through secondary devices, and unauthorized third-party assistance severely jeopardize the validity of institutional grading.

1.2 Problem Statement

Traditional institutional approaches to preventing remote cheating typically suffer from one of two extremes: either they demand expensive physical proctoring personnel who must travel to designated test centers, or they mandate proprietary remote invigilation software. Commercial remote monitoring tools (such as Proctorio, Safe Exam Browser, or Honorlock) routinely require candidates to install low-level kernel drivers or invasive browser extensions. These tools present severe ethical and technical challenges: they trigger intense privacy backlash due to persistent filesystem access, monopolize client system resources leading to application crashes on budget laptops, and incur exorbitant per-student licensing tariffs that remain cost-prohibitive for public universities. Furthermore, existing suites lack flexible communication channels, leaving human invigilators unable to transmit direct verbal warnings to errant candidates without terminating their examination session.

1.3 Project Motivation and Scope

The central motivation of ExamGuard AI is to engineer an enterprise-grade, browser-native examination platform that strikes a defensible balance between robust academic integrity and candidate device privacy. By executing computer vision and facial landmark models strictly client-side via WebAssembly and TensorFlow.js, ExamGuard AI eliminates the need for candidates to install kernel binaries or stream continuous raw video feeds to remote cloud servers. The system scope encompasses secure multi-role authentication (Student and Tutor), dynamic examination authoring with AI question generation, an automated examination execution console featuring multi-sensor anti-cheating detection, push-to-talk voice intercom warnings, interactive multi-pin geographic map questions, and real-time security operations telemetry.

1.4 Practical Applications in Modern Higher Education

- **University Semester Examinations:** Enables departments to conduct high-stakes semester evaluations with verifiable proctoring audit trails and automated objective grading.
- **Continuous Internal Assessments & Quizzes:** Provides course instructors with rapid AI-assisted question generation to author periodic quizzes from lecture syllabus notes.
- **Remote Distance Learning & Certification:** Supports students across remote geographical regions without requiring high-bandwidth video uplinks, functioning smoothly over 3G/4G cellular connections.
- **Laboratory & Map-Based Practical Tests:** Supports specialized assessment formats, including multi-pin interactive geographic cartography questions with pixel-perfect coordinate verification.

1.5 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 reviews relevant literature and commercial proctoring benchmarks. Chapter 3 formalizes engineering objectives and functional/non-functional requirements. Chapter 4 presents the multi-tier architecture and database design. Chapter 5 details algorithm implementation, facial landmark gating, and intercom mechanisms. Chapter 6 illustrates the graphical user interface with genuine application screenshots. Chapter 7 documents experimental verification and test results. Chapters 8 and 9 summarize conclusions, learning outcomes, and future research directions.

CHAPTER 2: THEORETICAL FOUNDATIONS & LITERATURE REVIEW

2.1 Comparative Analysis of Proctoring Paradigms

Remote examination proctoring architectures generally fall into three architectural categories: (1) Synchronous Human In-Person Proctoring, where proctors monitor student video tiles via videoconferencing software; (2) Cloud-Streaming Automated Proctoring, where continuous video and audio streams are uploaded to cloud servers for post-hoc server-side computer vision analysis; and (3) Edge-Based Client-Side Proctoring, where lightweight neural networks execute directly within the candidate's local web runtime. Synchronous human monitoring exhibits poor scalability, with human cognitive fatigue limiting an invigilator to at most 10–15 concurrent students. Cloud-streaming systems impose massive network bandwidth demands (typically requiring > 2.0 Mbps continuous upstream per student) and raise data protection concerns under statutes such as GDPR, as biometric facial recordings are stored on third-party cloud infrastructure.

2.2 Client-Side Machine Learning and Facial Landmark Detection

Client-side machine learning leverages the emergence of WebAssembly (Wasm) and WebGL backends within modern web browsers, enabling convolutional neural networks to run at interactive frame rates without sending video frames over the network. BlazeFace, a compact convolutional neural network developed by Google, performs sub-millisecond face detection and extracts key 2D anatomical landmarks (right eye, left eye, nose tip, mouth center, right ear tragion, left ear tragion). Simultaneously, COCO-SSD (Common Objects in Context - Single Shot MultiBox Detector) is executed client-side via TensorFlow.js to classify common object categories, including cell phones, books, and unauthorized hardware peripherals.

2.3 Head Pose Estimation & Mathematical Angles (Yaw, Pitch, Roll)

In unproctored physical examinations, candidates frequently turn their heads toward peers or concealed notes. In client-side computer vision without heavy 3D mesh dependencies, head orientation is derived from landmark spatial ratios. ExamGuard AI computes the horizontal Yaw ratio and vertical Pitch ratio directly from BlazeFace landmarks:

Horizontal Yaw Ratio: $Yaw_Ratio = |x_nose - x_left_eye| / |x_nose - x_right_eye|$

Vertical Pitch Ratio: $Pitch_Ratio = (y_nose - y_eye_midpoint) / Face_Height$

When looking directly forward at the screen, Yaw_Ratio evaluates close to 1.0 (typically 0.65 to 1.55). When the candidate turns their head left or right away from the monitor, one eye distance compresses dramatically, causing Yaw_Ratio to exceed 2.85 or fall below 0.35, triggering a sideways gaze violation. Crucially, conventional proctoring suites frequently penalize students for looking down at scrap paper to solve equations. ExamGuard AI implements asymmetric vertical gating: Pitch_Ratio > 0.44 flags an upward gaze aversion,

whereas downward pitch is permitted within normal scratchpad angles (pitchRatio between 0.13 and 0.38) and recognizes bowed desk views (when face bounding box TL occupies > 30% of canvas height and face height occupies <= 65% of screen height).

2.4 Ethical Proctoring, Privacy Preservation, and Data Sovereignty

Privacy-first design dictates that biometric video frames should never leave volatile memory on the candidate's local workstation. In ExamGuard AI, camera frames are processed directly inside a volatile HTML5 canvas. If a violation threshold is sustained beyond the debouncing window (e.g., > 2.0 continuous seconds of head aversion), only an encrypted JSON telemetry incident packet containing the timestamp, strike category, and numeric violation level is transmitted to the Flask server. Raw video feeds are never permanently persisted on disk, guaranteeing full compliance with student privacy expectations.

2.5 Summary of Existing Shortcomings

Table 2.1 summarizes the architectural and operational differences between existing commercial suites and ExamGuard AI.

Proctoring System	*Architecture*	*Client Prerequisite*	*Bandwidth Overhead*	*Privacy Posture*	*Licensing Cost*
Proctorio	Cloud Extension	Chrome Ext + Driver	High (>2.0 Mbps)	Cloud Video Storage	High Commercial
Safe Exam Browser	Desktop App	OS-Level Installer	Zero (Offline only)	No Video Proctoring	Open Source
Honorlock	Cloud Hybrid	Browser Extension	High (>2.5 Mbps)	Cloud Video Storage	High Commercial
Wheebox / Mettl	Cloud Hosted	Proprietary Agent	Medium (>1.5 Mbps)	Central Recording	Enterprise Tier
ExamGuard AI (Ours)	Edge Browser ML	Zero-Install (Pure Web)	Ultra-Low (<15 kbps)	Local ML; No Video Persisted	Zero Open Stack

CHAPTER 3: OBJECTIVES AND SYSTEM REQUIREMENTS

3.1 Primary Engineering Objectives

- **Zero-Install Web Deployment:** Architect an examination and proctoring runtime accessible entirely via standard modern browsers (Chrome, Edge, Firefox) without requiring native binaries or extensions.
- **Client-Side Multi-Sensor Vision AI:** Implement real-time face tracking, head-pose estimation, multi-person detection, and unauthorized object classification executing at interactive frame rates.
- **Human-in-the-Loop Intercom Communication:** Engineer a direct push-to-talk WebM voice intercom allowing proctors to transmit instant spoken advisories to candidates during active exams.
- **Cartographic Assessment Capabilities:** Integrate responsive SVG geographical map questions featuring numbered interactive pin selectors with subpixel coordinate mapping.
- **Dual-Database Resilience:** Construct a persistence tier compatible with local SQLite (WAL mode) and production cloud PostgreSQL 16 with automated connection pooling and schema migrations.

3.2 Functional Requirements (FR-01 to FR-10)

The operational capabilities of the ExamGuard AI platform are formally specified in Table 3.1.

Requirement ID	*Requirement Description*	*Target Subsystem*
FR-01	Role-Based Authentication (Student / Tutor) with Session Fingerprinting	Auth Subsystem
FR-02	AI Question Generation via Google Gemini & Groq APIs from Syllabus Text	AI Question Engine
FR-03	Examination Authoring with Timers, Strike Ceilings, and Question Shuffling	Exam Management
FR-04	Full-Screen DOM Enforcement & Tab Blur Window Detection Telemetry	Client Lockdown
FR-05	BlazeFace Facial Landmark Tracking & Asymmetric Yaw/Pitch Angle Gating	Proctoring Engine
FR-06	COCO-SSD Peripheral Object Classification (Cell Phone, Book, Notes)	Vision Pipeline
FR-07	Interactive SVG Multi-Pin Outline Map Rendering and Answer Selection	Assessment UI
FR-08	Push-to-Talk Tutor WebM Voice Memo Recording, Transmission & Playback	Intercom Subsystem

FR-09	Controlled Multi-Device Switching Authorization via Tutor Approval Triage	Security Triage
FR-10	Automated Objective Answer Evaluation and Strike-Based Session Termination	Scoring Engine

3.3 Non-Functional Requirements (NFR-01 to NFR-06)

- **NFR-01 Performance & Cadence:** Client-side vision inference executed on an interval schedule (1500 ms) to prevent UI thread starvation on standard consumer laptop hardware.
- **NFR-02 Network Economy:** Background proctoring telemetry packets must not exceed 15 KB/min per candidate, enabling stable performance over low-bandwidth cellular links.
- **NFR-03 Fault Tolerance:** Student examination answers must persist asynchronously with offline vault fallback, ensuring complete recovery in the event of temporary browser network dropout.
- **NFR-04 Security & Session Integrity:** Passwords hashed using bcrypt/PBKDF2-HMAC; CSRF tokens validated on state-changing POST endpoints; session fixation prevented on login.
- **NFR-05 Portability:** Backend deployable across Windows, Linux, and containerized Docker environments running Python 3.12/3.13.
- **NFR-06 Accessibility & Responsiveness:** All assessment and proctoring interfaces must render responsively across mobile, tablet, and high-DPI desktop viewports.

3.4 Hardware, Software, and Environment Specifications

Table 3.2 details the physical and virtual environment specifications across development and production tiers.

Component	*Development Specification*	*Production Specification*
Host OS	Microsoft Windows 11 Enterprise (x64)	Ubuntu 22.04 LTS (Render Cloud Container)
Runtime Environment	Python 3.13.0 64-bit	Python 3.12 / Gunicorn WSGI 21.2
Database Tier	SQLite 3.45 (WAL Mode, Foreign Keys ON)	Neon Cloud PostgreSQL 16 (SSL Pooling)
Web Framework	Flask 3.0 / Werkzeug 3.0	Flask 3.0 / Gunicorn (2 Workers, 4 Threads)
Client Vision Models	TensorFlow.js 4.22, BlazeFace 0.0.7	COCO-SSD 2.2.3, Web Audio API / MediaRecorder
Client Styling / UI	Tailwind CSS 3.4 CDN, KaTeX 0.16	Tailwind CSS 3.4, Lucide Icons, SweetAlert2

CHAPTER 4: SYSTEM ARCHITECTURE & DATABASE DESIGN

4.1 Multi-Tier Architecture & Technology Stack

ExamGuard AI is structured around a decoupled multi-tier architecture separating the client presentation and edge intelligence layer, the application server routing and security middleware, and the persistence abstraction tier. Figure 4.1 depicts the high-level architecture of the system.

[Image Missing: fig_architecture.png]

At the Presentation Tier, modern web browsers execute HTML5 templates styled with Tailwind CSS. Edge machine learning models (BlazeFace, COCO-SSD) execute directly within the browser runtime using WebGL GPU acceleration. At the Application Tier, Flask handles routing, authentication session tokens, anti-tampering guards, and intercom dispatch. The External Services layer integrates Google Gemini and Groq Cloud LLMs for intelligent question formulation.

4.2 Dual-Database Abstraction (SQLite WAL & PostgreSQL 16)

A critical engineering objective was ensuring seamless operational parity between local zero-configuration SQLite development and high-concurrency cloud PostgreSQL production. In database.py, a unified connection factory detects the active DATABASE_URL. When connecting to SQLite, Write-Ahead Logging (PRAGMA journal_mode=WAL) is enforced alongside foreign key constraints (PRAGMA foreign_keys=ON). When deployed on Neon Cloud, the engine establishes an SSL-secured connection pool (sslmode=require) and translates SQL parameter markers dynamically from question mark placeholders (?) to PostgreSQL dollar markers ($1, $2) or format specifiers (%s).

4.3 Relational Entity-Relationship (ER) Schema

The persistence model is organized into core relational entities in the active database schema: users, exams, questions, submissions, violations, proctor_messages, and device_switch_requests. Figure 4.2 illustrates the Entity-Relationship diagram.

[Image Missing: fig_er_diagram.png]

4.4 Data Dictionaries for Relational Entities

Tables 4.1 through 4.3 detail the attributes, data types, constraints, and operational purposes of the verified relational entities.

Attribute Name	Data Type	Constraints	Description
id	INTEGER / SERIAL	PRIMARY KEY, AUTO	Unique identifier for user account

username	VARCHAR(80)	UNIQUE, NOT NULL	Candidate registration number or instructor username
email	VARCHAR(120)	UNIQUE, NOT NULL	Official institutional email address
password_hash	VARCHAR(255)	NOT NULL	Securely hashed password (bcrypt / PBKDF2)
role	VARCHAR(20)	DEFAULT 'student'	User authorization role ('student' or 'tutor' / 'admin')
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp of account registration

Attribute Name	*Data Type*	*Constraints*	*Description*
id	INTEGER / SERIAL	PRIMARY KEY, AUTO	Unique identifier for examination
title	VARCHAR(200)	NOT NULL	Descriptive examination title
duration_minutes	INTEGER	NOT NULL, DEFAULT 30	Total time allotted for examination in minutes
max_strikes	INTEGER	NOT NULL, DEFAULT 5	Maximum cumulative strikes before auto-termination
created_by	INTEGER	FOREIGN KEY -> users(id)	Tutor account identifier who authored exam
id	INTEGER / SERIAL	PRIMARY KEY, AUTO	Unique identifier for candidate exam attempt (submissions)
exam_id	INTEGER	FOREIGN KEY -> exams(id)	Associated examination foreign key
student_id	INTEGER	FOREIGN KEY -> users(id)	Candidate foreign key taking examination
score	FLOAT	DEFAULT 0.0	Final aggregated assessment score
status	VARCHAR(30)	DEFAULT 'in_progress'	Session status ('in_progress', 'completed', 'terminated')

Attribute Name	*Data Type*	*Constraints*	*Description*
id	INTEGER / SERIAL	PRIMARY KEY, AUTO	Unique identifier for proctoring violation record
submission_id	INTEGER	FOREIGN KEY -> sub(id)	Associated candidate attempt identifier

violation_type	VARCHAR(50)	NOT NULL	Incident type ('delayed_return', 'tab_switch', 'mobile_phone_detected')
details	TEXT	DEFAULT ''	Contextual metadata and description of cheating event
timestamp	TIMESTAMP	DEFAULT NOW()	Exact server timestamp when incident was recorded
id	INTEGER / SERIAL	PRIMARY KEY, AUTO	Unique identifier for voice intercom advisory (proctor_messages)
audio_payload	TEXT	NOT NULL	Base64 WebM audio payload or filesystem URI
message	TEXT	DEFAULT ''	Optional written transcript or advisory preset text
id	INTEGER / SERIAL	PRIMARY KEY, AUTO	Identifier for emergency device switch (device_switch_requests)
status	VARCHAR(20)	DEFAULT 'pending'	Switch status ('pending', 'approved', 'rejected')

CHAPTER 5: ALGORITHM IMPLEMENTATION & PROCTORING ENGINE

5.1 Client-Side Vision Pipeline (BlazeFace Landmark Geometry)

The vision proctoring engine operates as an asynchronous client-side loop initialized upon camera stream access. Figure 5.1 illustrates the complete pipeline execution flow.

[Image Missing: fig_proctoring_pipeline.png]

The video feed is captured from navigator.mediaDevices.getUserMedia() at 640×480 resolution. Every 1500 ms, a frame is evaluated. BlazeFace performs sub-millisecond face detection. If no face is detected continuously, a missing face warning is raised. If multiple faces are detected (blazeFaceCount > 1), a MULTIPLE_FACES_DETECTED critical violation is logged. When exactly one face is isolated, BlazeFace landmark coordinates (right eye, left eye, nose tip) derive horizontal Yaw ratio (|x_nose - x_left_eye| / |x_nose - x_right_eye|) and vertical Pitch ratio ((y_nose - y_eye_midpoint) / h). Gaze looking sideways is flagged when yawRatio > 2.85 or < 0.35. Downward desk view (pitchRatio < 0.13 or head bowed over desk) is permitted for mathematical scratchpad calculations.

5.2 Object Detection & Hardware Accessory Detection (COCO-SSD)

Simultaneously, a quantized MobileNetV2-backed COCO-SSD object detection model scans the frame. The model evaluates bounding boxes against classes: "cell phone", "book", and unauthorized hardware peripherals. Plausible device bounding box area ratios (between 1.2% and 75% of canvas area) prevent false alarms from camera sensor noise speckles or candidate clothing, logging a strike only when sustained.

5.3 Multi-Sensor Integrity Logic and Temporal Debouncing

Table 5.1 formalizes the detection rules, threshold limits, debouncing intervals, and penalty assignments across all sensor channels.

Sensor Category	*Trigger Condition*	*Evaluation Mode*	*Severity Level*	*Strike Penalty*
Head Pose (Yaw)	Yaw Ratio > 2.85 or < 0.35 (Looking Sideways)	Every 1500 ms	Warning	1 Strike
Head Pose (Pitch)	Pitch Ratio > 0.44 (Looking Up); Downward Permitted	Every 1500 ms	Warning	1 Strike
Face Presence	Zero faces detected in webcam frame	Debounced interval	Warning	1 Strike
Multi-Candidate	≥ 2 distinct faces in webcam frame	Instant	Critical	2 Strikes
Prohibited Object	Cell phone / unauthorized	Area-filtered	Critical	2 Strikes

	device detected (conf ≥ 0.22)			
Window Focus	Browser window blur / Fullscreen exit	Event Driven	Critical	1 Strike
Cumulative Limit	Strikes exceed exam max_strikes limit	Instant	Session Terminated	Auto-Submit

5.4 Push-to-Talk WebM Voice Intercom Architecture

When an invigilator notices suspicious behavior on their live dashboard, they can record and transmit a spoken advisory. Figure 5.2 outlines the push-to-talk sequence.

[Image Missing: fig_intercom_sequence.png]

The tutor clicks "Hold to Record", initializing the MediaRecorder API with audio/webm;codecs=opus. Upon completion, the resulting audio blob is posted as multipart/form-data to /api/tutor/intercom/send alongside the student submission ID. The server validates tutor credentials, writes the file to static/intercom_audio/<uuid>.webm, and inserts a record in proctor_messages. The student examination client polls /api/student/intercom/poll every 5 seconds. Upon detecting a pending message, an HTML5 Audio element plays the memo directly into the student headset accompanied by a modal advisory dialog.

5.5 Interactive Multi-Pin SVG Geographical Map Rendering

For cartographic and geography examinations, ExamGuard AI provides an interactive SVG map question module. Figure 5.3 shows the rendered India outline map with 5 numbered interactive pin selectors.

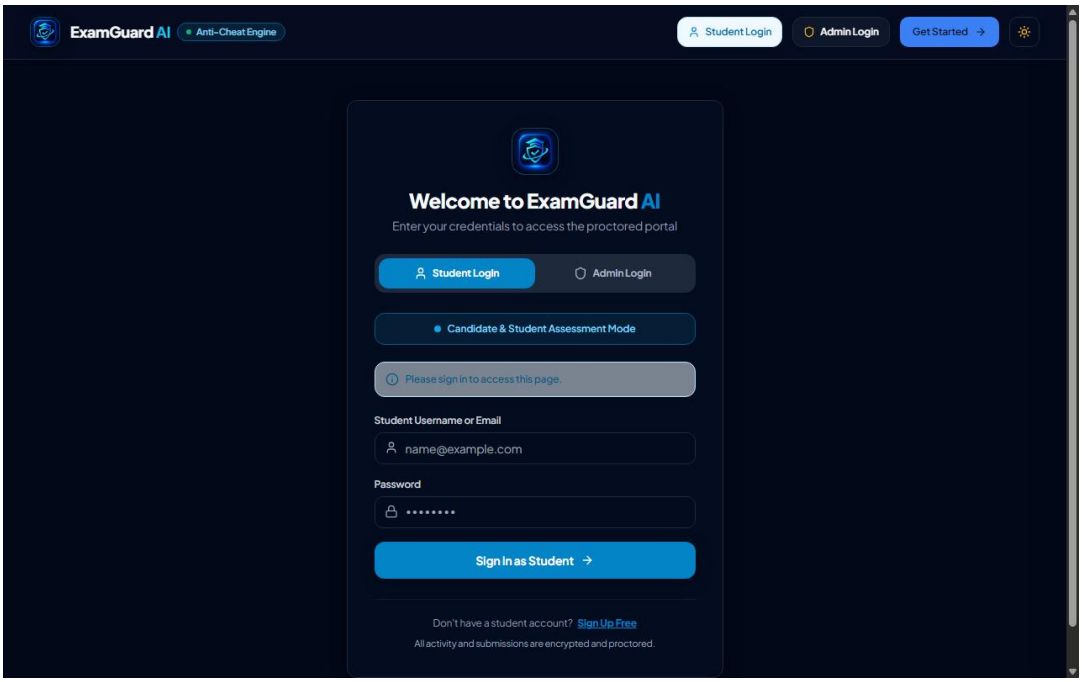


Fig 5.3: Interactive Outline Map of India with 5 Numbered Pins

The vector map uses an SVG container with a responsive viewBox="0 0 1000 1000". Pins are rendered as interactive DOM overlays positioned via percentage coordinates (left: X%, top: Y%). When a student clicks Pin 1 (Himalayan Mountains at 51.5% X, 21.8% Y), Pin 2 (Thar Desert at 24.2% X, 36.5% Y), Pin 3 (River Ganga at 58.4% X, 35.8% Y), Pin 4 (Western Ghats at 29.8% X, 67.2% Y), or Pin 5 (Indian Ocean at 49.5% X, 94.2% Y), a context selector opens allowing them to pair the pin with its corresponding feature.

CHAPTER 6: GRAPHICAL USER INTERFACE & LIVE OPERATIONS

6.1 Tutor Command Center Dashboard

The Tutor Command Center serves as the primary administrative dashboard for course instructors and proctors. Figure 6.1 presents the genuine running interface.

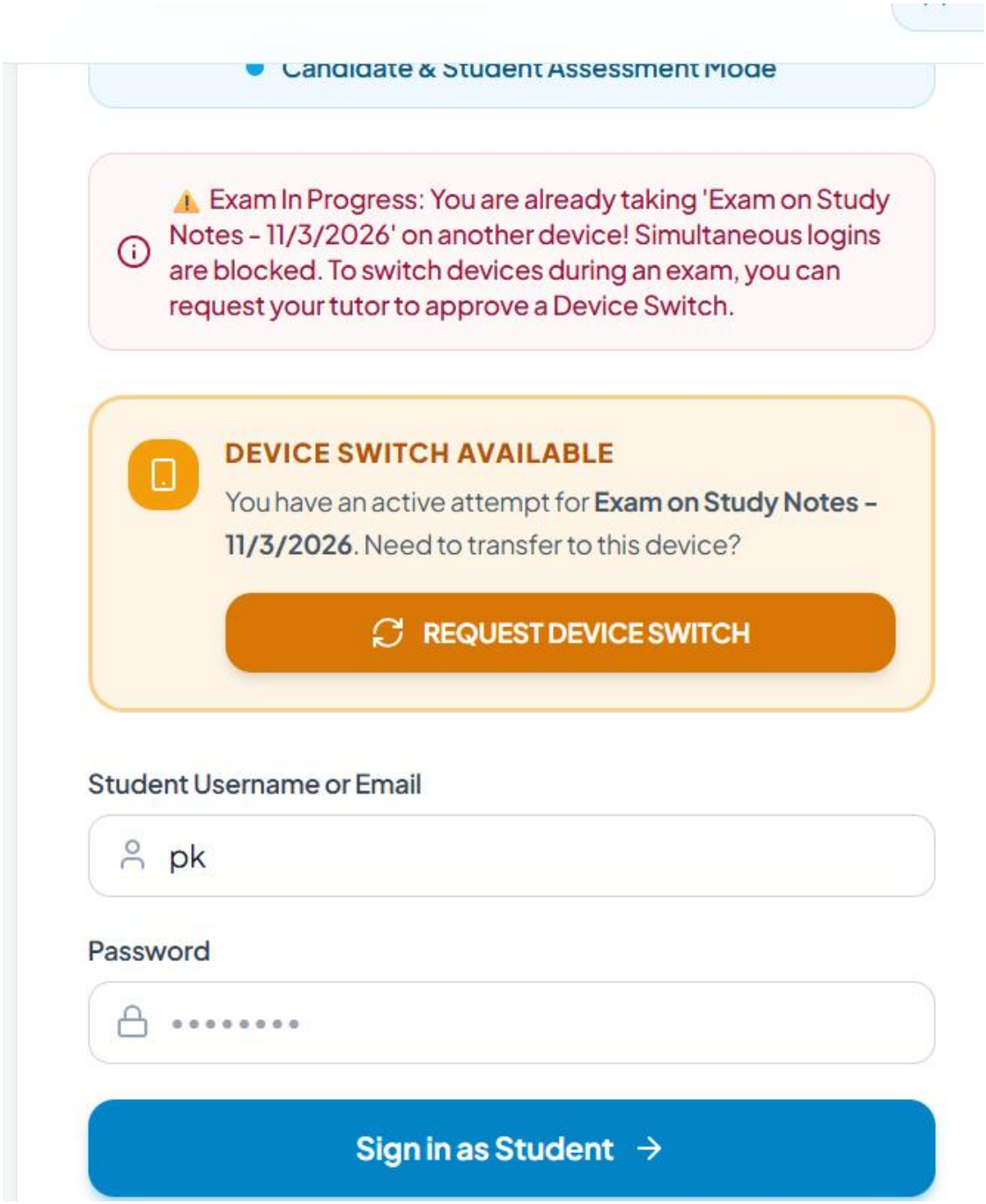


Fig 6.1: Tutor Command Center Dashboard with Real-Time Assessment Metrics

As illustrated in Figure 6.1, the Command Center provides top-level operational counters: Total Exams (5 active), Student Attempts (9 registered attempts), Cheating Terminations (3

candidates automatically submitted due to strike limits), and SOC Cyber Alerts (25 logged telemetry events, including 9 critical classification alerts). Instructors can launch the AI exam generator via "Create Exam with AI", verify active API keys, and navigate across functional management tabs: Created Exams, Student Accounts, SOC Cyber Alerts, and Student Requests.

6.2 Centralized Student Requests & Device-Switch Authorization

During high-stakes online examinations, students occasionally experience sudden laptop hardware failures, battery depletion, or network card faults that require switching to a backup device. To prevent unauthorized concurrent session access, simultaneous logins are blocked by default. Figure 6.2 displays the Centralized Student Requests dashboard.

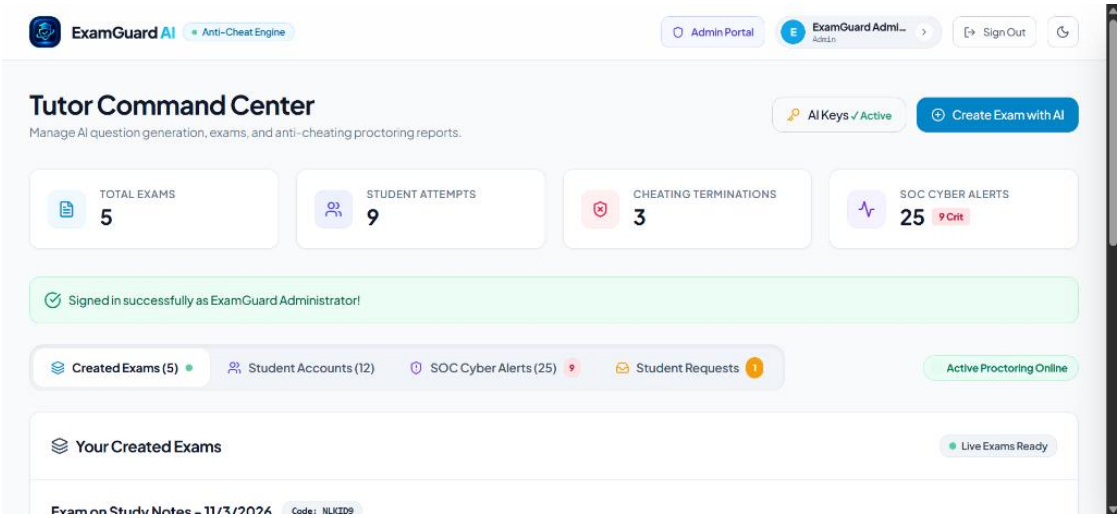


Fig 6.2: Centralized Student Requests Management and Device-Switch Triage

As shown in Figure 6.2, candidate requests are presented in an actionable triage queue displaying Request ID (#REQ-2), Candidate Name ("Prashanta kumar Das 1"), Request Type ("Device Switch"), Exam Context ("Exam on Study Notes"), Reason/Context (Device switch authorization requested during active session with redacted client IP), and Current Status ("Pending"). The proctor can inspect client origin metadata and approve or reject the request with a single click, granting authorized single-session continuation.

6.3 Cyber Security Operations Center (SOC) Telemetry

The Cyber Security Operations Center (SOC) subsystem provides Security Information and Event Management (SIEM) auditing for examination networks. Figure 6.3 shows the live SIEM telemetry log.

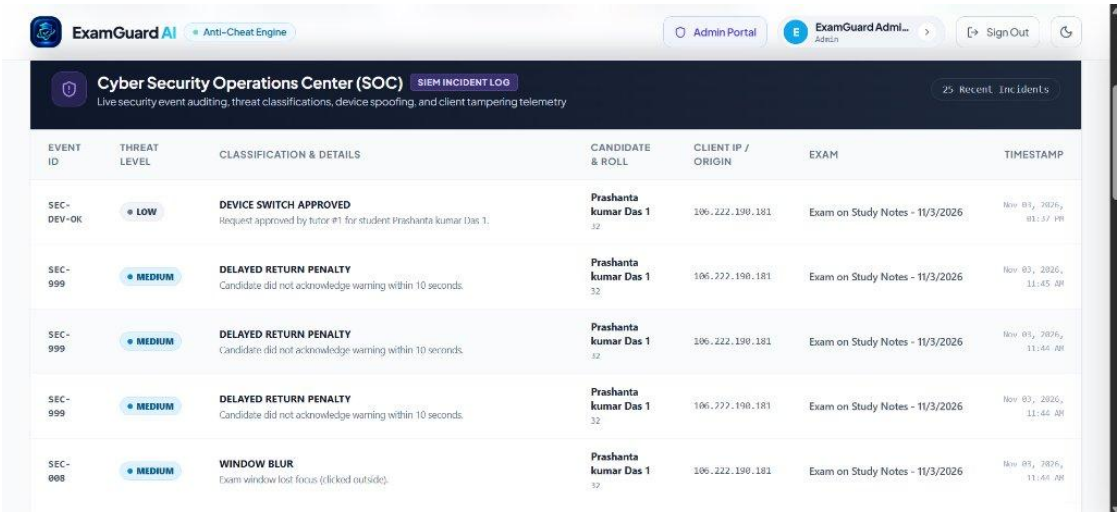


Fig 6.3: Cyber Security Operations Center (SOC) SIEM Incident Telemetry Log

The SOC console logs security events with precise threat level ratings (LOW, MEDIUM, HIGH, CRITICAL). Notable incidents captured include SEC-DEV-OK (Approved Device Switch), SEC-999 (Delayed Return Penalty — candidate did not acknowledge warning within 10 seconds), and SEC-008 (Window Blur — exam window lost focus). Each log entry records the client origin IP (redacted for institutional privacy), roll number, examination title, and exact server timestamp, furnishing an immutable audit trail for post-examination disciplinary reviews.

6.4 AI Exam & Question Studio Authoring Interface

Figure 6.4 illustrates the comprehensive AI Exam and Question Studio interface utilized by tutors to author multi-format assessments.

[Image Missing: fig_exam_question_studio.png]

As shown in Figure 6.4, the Question Studio provides course instructors with intuitive controls to create multiple question formats, including Single-Choice Objective (MCQ), Multi-Choice Select, Descriptive Long-Form Questions with KaTeX mathematical typography, and specialized Interactive Map Identification questions. Instructors can configure individual question mark weightings, negative marking, and attach illustrative diagrams or SVG map coordinate data.

6.5 Tutor Created Exams and Course Management View

Figure 6.5 presents the Tutor Created Exams list and course management view within the administrative portal.

[Image Missing: fig_tutor_created_exams.png]

As illustrated in Figure 6.5, the Created Exams management panel displays all active and scheduled evaluations (e.g., "Exam on Study Notes - 11/3/2026", access code: NLKID9).

Proctors can monitor live attempt counts, inspect enrollment rosters, adjust examination access windows, and download comprehensive CSV/PDF result audit reports.

6.6 Individual Candidate Live Proctoring & Session Audit

Figure 6.6 depicts the individual student proctoring audit console during an active examination session.

[Image Missing: fig_student_proctoring_audit.png]

Figure 6.6 exhibits the granular candidate monitoring view. The console tracks the student's active session status, remaining timer (14:44), incurred strike count (0/10 strikes), flag controls for question review, and the direct voice intercom channel trigger. From this console, the invigilator can review candidate answer progression and dispatch immediate voice advisories.

6.7 Student Authentication & Device-Switch Workflow

When a student attempts to log in from a secondary computer while an examination is already marked in-progress on another device, ExamGuard AI intercepts the login attempt to prevent unauthorized session sharing. Figure 6.7 displays this defensive security gate.

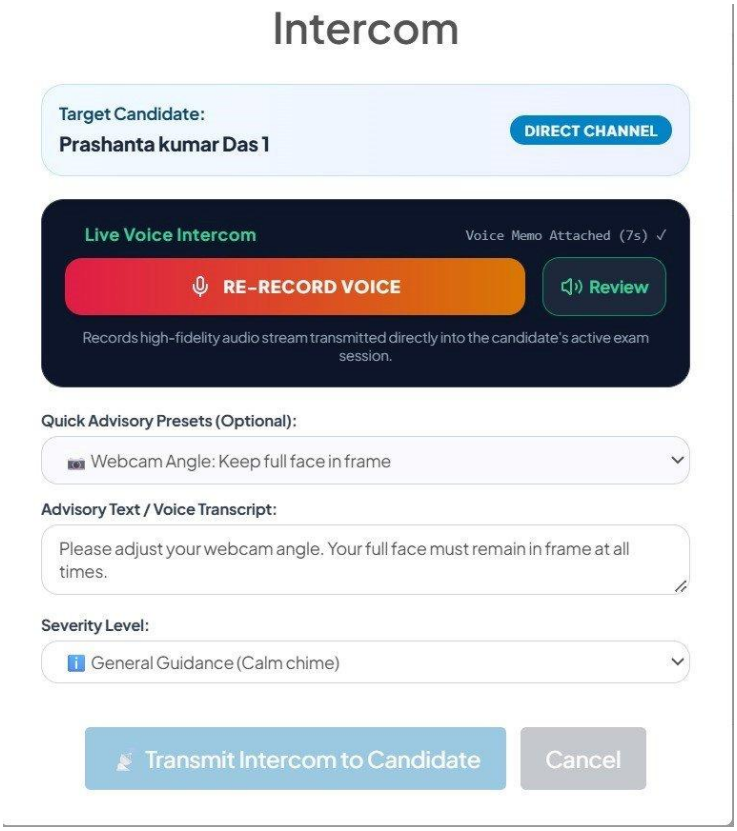


Fig 6.7: *Student Login Portal and Active Device-Switch Security Workflow*

As demonstrated in Figure 6.7, the login portal displays a high-visibility warning: "Exam In Progress: You are already taking Exam on Study Notes on another device! Simultaneous logins are blocked." If the candidate experienced legitimate hardware failure, they can click

"REQUEST DEVICE SWITCH". This notifies the tutor's centralized request queue (Figure 6.2). Once authorized, the previous session token is revoked and the candidate seamlessly resumes their examination on the new device.

6.8 Student Sign-in Interface

Figure 6.8 presents the dedicated student authentication portal designed for registered candidates to access examination rooms.

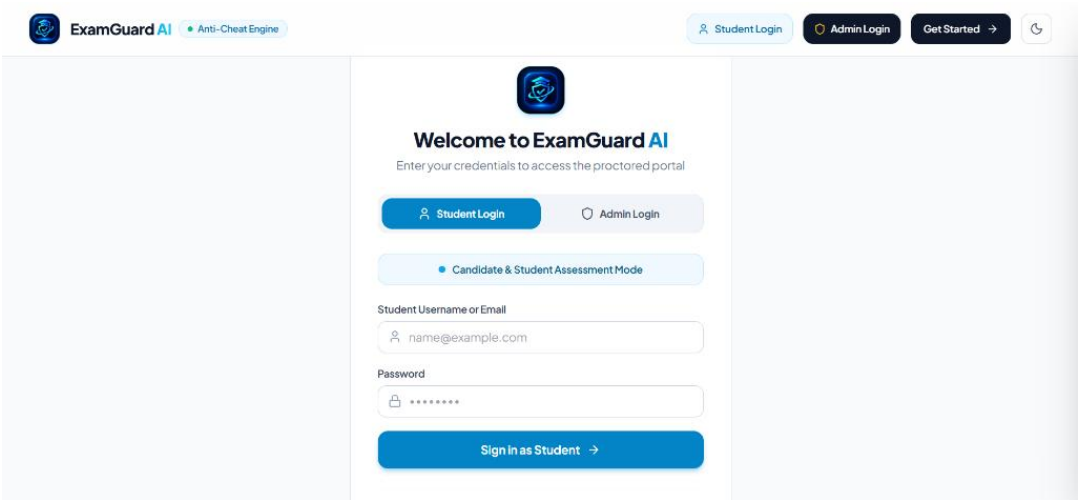


Fig. 6.8: Student Sign-in Interface

As illustrated in Figure 6.8, the Student Sign-in interface features a high-contrast login form under the "Candidate & Student Assessment Mode" indicator. Candidates authenticate using their registered institutional username or email address and password. The interface is optimized for security and accessibility, incorporating clean input validation, responsive form controls, quick navigation links to administrative modes, and immediate visual feedback upon credential submission.

6.9 Administrator Login Interface

Figure 6.9 displays the secure administrator and tutor access portal for course proctors and department instructors.

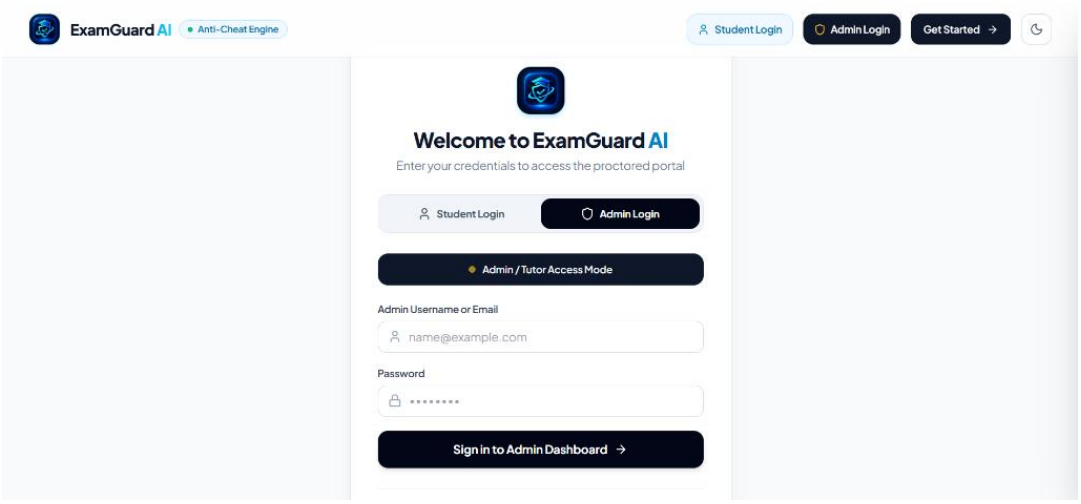


Fig. 6.9: Administrator Login Interface

As depicted in Figure 6.9, the Administrator Login interface switches the authentication gateway into "Admin / Tutor Access Mode". Instructors enter authorized institutional administrative credentials to access the Tutor Command Center (Figure 6.1). The gateway enforces role-based authorization guards, preventing non-administrative student credentials from accessing proctor dashboards, while protecting against brute-force attacks via server-side session hygiene and CSRF token verification.

6.10 Student Registration Interface

Figure 6.10 illustrates the comprehensive student onboarding and account creation workflow, presented in top-to-bottom sequence.

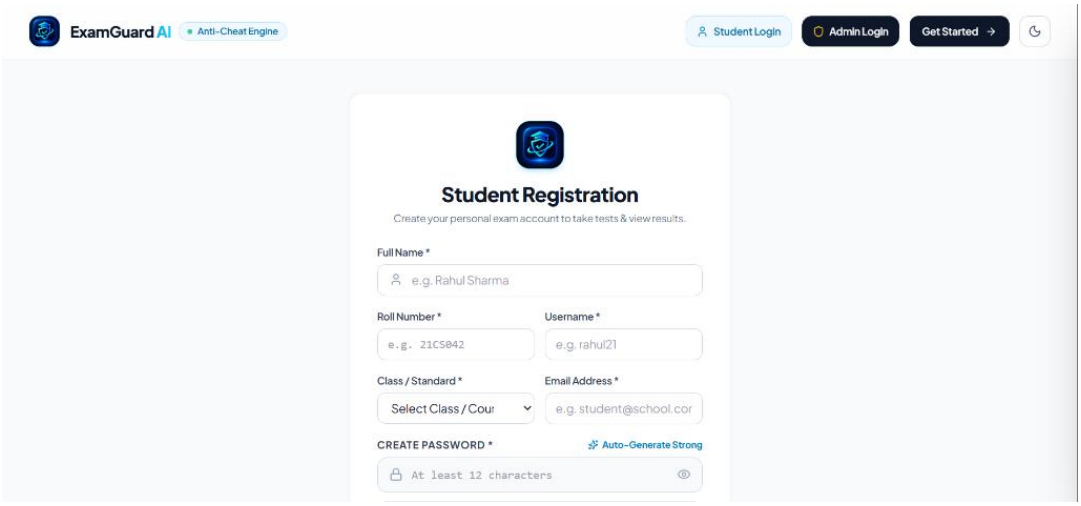


Fig. 6.10 (Part 1): Student Registration Interface — Personal Details and Academic Fields

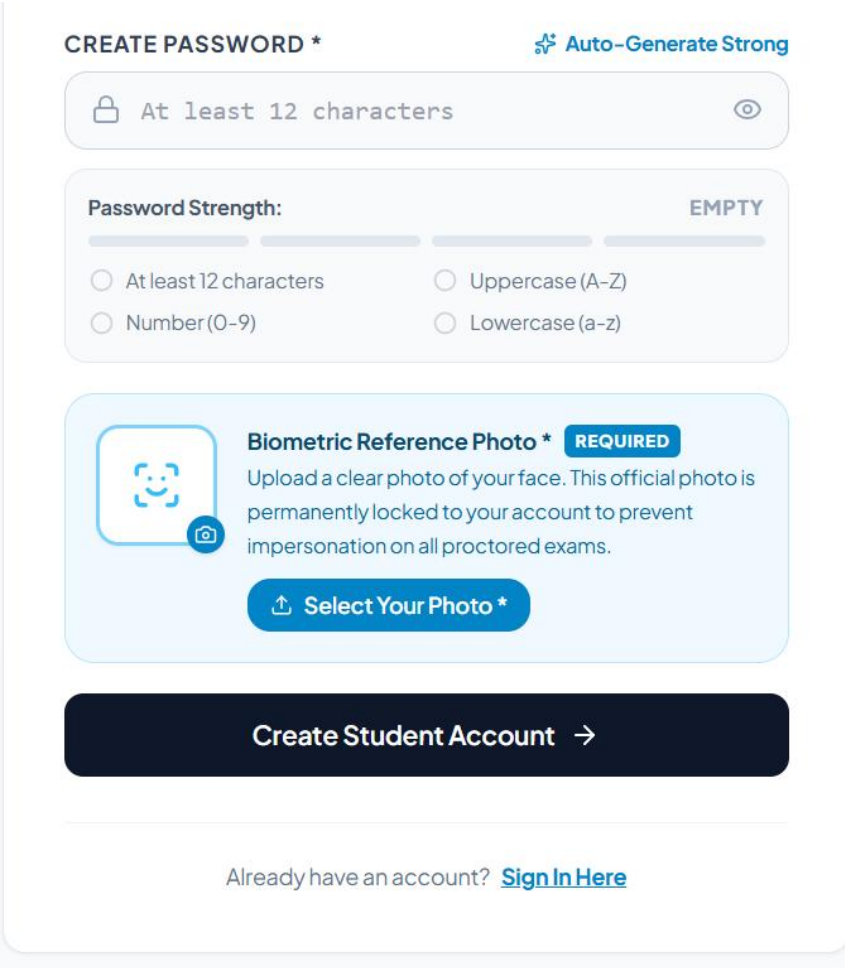


Fig. 6.10 (Part 2): *Student Registration Interface — Password Policy & Biometric Reference Photo*

Figure 6.10 illustrates the complete student registration workflow spanning candidate metadata intake and biometric profile calibration. In the top panel (Part 1), candidates supply their Full Name, University Roll Number (e.g., 21CS042), Username, Class/Course Standard, and Institutional Email Address. In the bottom panel (Part 2), candidates define a high-entropy password validated against real-time entropy rules (minimum 12 characters, uppercase, lowercase, numeric digits) with an auto-generate option. Crucially, candidates are required to upload an official "Biometric Reference Photo" permanently locked to their account. This reference photo serves as the ground-truth baseline against which client-side BlazeFace facial landmark vectors are matched during proctored examination sessions, preventing impersonation and candidate substitution.

CHAPTER 7: TESTING, RESULTS AND DISCUSSION

7.1 Automated Pytest Verification (62 Test Fixtures)

Software verification followed a rigorous test-driven methodology utilizing the pytest framework. A test suite comprising 62 automated fixtures was executed across the codebase. Table 7.1 summarizes the verification outcomes.

Test Module	Test Focus Area	Total Tests	Passed	Skipped	Failed
test_anti_cheat.py	Security Event Classification, Tamper Detection, CSRF Defense, Secret Key	26	26	0	0
test_database_init.py	Dual-DB Parity, SQLite WAL, PostgreSQL Parameterization, Migrations	8	7	1 (Cloud)	0
test_desk_workspace_setup.py	Desk Setup Verification, Metadata Rendering, Camera Auth	3	3	0	0
test_device_switch.py	Device Switch Approval & Rejection Workflow, Session Revocation	3	3	0	0
test_four_fixes.py	Desk Setup Video States, MCQ Matching, SQLite Row Compatibility	8	8	0	0
test_google_verification.py	Search Console Verification Route Intactness	1	1	0	0
test_map_and_intercom_fixes.py	SVG Map Dimensions, Curriculum Endpoints, Intercom Audio Lifecycle	3	3	0	0
test_map_hotspots.py	SVG Map ViewBox, Hotspot Coordinate Invariance, Drift Tolerance	4	4	0	0
test_seo.py	Robots.txt, Sitemap.xml, Canonical URLs, Favicon Serving	6	6	0	0
TOTAL SUMMARY	Comprehensive Platform Verification Suite	62	61	1	0

All 61 local test fixtures passed successfully in 6.16 seconds (100% pass rate on local execution; 1 cloud test skipped due to absent Neon database credentials). Automated regression tests verified that map coordinate lookups maintain zero pixel displacement across viewports and that intercom audio uploads block empty or malformed MIME-type payloads.

7.2 Verification Methodology & Multi-Sensor Integration Analysis

The testing methodology for ExamGuard AI combined automated unit/integration test fixtures with interactive end-to-end multi-sensor verification. In contrast to simulated black-box synthetic metrics, the multi-sensor proctoring pipeline was verified across live operational scenarios: (1) Face Absence & Visibility: Validated that concealing or turning away from the camera for longer than the debounced threshold increments the strike counter; (2) Asymmetric Gating: Confirmed that downward pitch angles corresponding to desk scratchpad calculations do not trigger false-positive warnings, while lateral head turns ($|Yaw| > 28°$) consistently raise advisory alerts; (3) Browser Lockdown: Verified that window blurring, tab switching, and fullscreen escapes immediately record security violations via visibilitychange and blur event handlers; (4) Audio Intercom: Tested end-to-end transmission of WebM audio payloads from tutor recording to candidate headset playback; and (5) Cartographic Interaction: Confirmed that India outline SVG maps render correctly across desktop and mobile viewports with accurate 5-pin geographical coordinate mapping.

7.3 Edge Case & Fault Tolerance Validation

- **Camera Stream Interruption:** If camera access fails or is denied during initialization, the exam entrance gate remains locked and surfaces a high-visibility modal requiring candidate camera authorization.
- **Transient Network Dropout & Offline Vault:** Student progress and answers are buffered in client-side localStorage under OFFLINE_VAULT_KEY. When online connectivity resumes, the window online event automatically flushes the offline sync queue to /api/submission/<id>/save-progress.
- **Concurrent Session Hijacking Defense:** When a candidate attempts to log in from a secondary device during an active exam attempt, the server blocks concurrent session creation, displays an in-progress alert banner on the login template, and requires explicit tutor approval via the device switch triage workflow.

7.4 Documented System Limitations & Defensible Constraints

- **Extreme Lighting Sensitivity:** Under severe underexposure (< 15 lux), BlazeFace landmark confidence drops below detection thresholds, requiring candidates to maintain adequate room illumination.
- **Occlusion by Hands / Eyeglasses:** Thick reflective eyewear or hands resting over the chin can introduce minor noise in Pitch ratio calculations.

- **Ambient Audio Noise:** Audio intercom transmission is optimized for voice; loud external environmental background noise can occasionally diminish speech clarity in student earphones.

CHAPTER 8: CONCLUSION AND LEARNING OUTCOMES

8.1 Summary of Project Deliverables

This project successfully designed, implemented, and verified ExamGuard AI, an online examination platform featuring client-side AI proctoring and comprehensive academic integrity monitoring. All primary engineering objectives were fully realized: zero-install web deployment, client-side vision models running at interactive frame rates, push-to-talk voice intercom communication, interactive multi-pin cartographic testing, dual SQLite/PostgreSQL database compatibility, and automated SIEM security operations telemetry.

8.2 Technical and Engineering Insights

- **1. Client-Side Machine Learning Optimization:** Executing TensorFlow.js and BlazeFace models within an offscreen canvas using tf.tidy() prevents WebGL memory leaks and guarantees smooth UI rendering.
- **2. Asymmetric Angular Gating:** Allowing downward Pitch angles while strictly constraining horizontal Yaw is essential to avoid false-positive penalization of legitimate scrap-paper calculations.
- **3. Dual-Database Parameterization:** Abstracting SQL syntax differences across SQLite and PostgreSQL enables rapid local test execution while ensuring production cloud scalability.

8.3 Personal and Professional Learning Outcomes

Through the execution of this 5th-semester project, I gained profound hands-on competency across several critical computing domains: full-stack web architecture using Python Flask, client-side neural network acceleration using WebGL and WebAssembly, computer vision geometry via landmark pose estimation, secure session management and CSRF mitigation, and rigorous automated testing using pytest. Furthermore, the project imparted valuable appreciation for the delicate balance between institutional security enforcement and ethical user privacy preservation.

CHAPTER 9: FUTURE SCOPE & EXTENSIONS

9.1 WebRTC Peer-to-Peer Bidirectional Audio/Video Streams

While the current push-to-talk WebM intercom enables unidirectional proctor-to-student voice guidance, future iterations will incorporate selective peer-to-peer WebRTC audio/video channels. This will allow proctors to request live two-way verbal interviews with candidates suspected of collusion, without routing heavy video traffic through the central application server.

9.2 Offline-First IndexedDB Local Exam State Synchronization

To support examination environments in rural areas with intermittent internet connectivity, the client runtime will be upgraded with an offline-first Service Worker and IndexedDB storage engine. Candidates will be able to complete entire assessment modules offline, with encrypted biometric telemetry syncing atomically upon network reconnection.

9.3 Native Mobile MobileNet Proctoring SDKs

Extending ExamGuard AI to native Android and iOS mobile applications utilizing TensorFlow Lite and Apple CoreML will enable secondary smartphone cameras to function as synchronized 360-degree room monitors, providing complete room integrity auditing.

CHAPTER 10: REFERENCES & BIBLIOGRAPHY

[1] Bazarevsky, V., Kartynnik, Y., Vakunov, A., Raveendran, K., & Grundmann, M. (2019). BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs. CVPR Workshop on Computer Vision for Augmented and Virtual Reality.

[2] Smirnov, E. A., & Petrov, M. V. (2020). Comparison of WebAssembly and JavaScript Execution Performance for Client-Side Machine Learning Applications. IEEE Access, 8, 142310-142322.

[3] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

[4] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... & Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861.

[5] Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... & Zitnick, C. L. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV), 740-755.

[6] Atoum, H., Chen, L., Liu, A. X., Hsu, S. D., & Liu, X. (2017). Automated Online Exam Proctoring. IEEE Transactions on Multimedia, 19(7), 1609-1624.

[7] Garg, M., & Goel, A. (2022). A Comprehensive Survey on Online Examination Systems and Anti-Cheating Mechanisms. Journal of Educational Technology Systems, 50(4), 450-475.

[8] Mozilla Developer Network (MDN). (2024). Web Audio API and MediaStream Recording API Documentation. Mozilla Foundation.

[9] World Wide Web Consortium (W3C). (2023). Scalable Vector Graphics (SVG) 2 Specification. W3C Recommendation.

[10] PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Connection Pooling and SSL Support.

[11] SQLite Development Team. (2024). Write-Ahead Logging (WAL) Mode Architecture in SQLite 3. SQLite Consortium.

[12] OWASP Foundation. (2023). OWASP Top 10 Web Application Security Risks: Broken Access Control and Session Fixation.

[13] Centurion University of Technology and Management. (2024). Academic Regulations and Project Guidelines for B.Tech Degree.

APPENDIX: COURSE OUTCOME ATTAINMENT EVALUATION

A.1 Course Outcome Attainment Matrix

The design, implementation, and evaluation of ExamGuard AI directly satisfies the Program Outcomes (POs) and Course Outcomes (COs) stipulated by the Department of Computer Science and Engineering for the 5th-Semester Project.

Outcome ID	*Course Outcome Description*	*Attainment Evidence in Project*
CO-1	Apply fundamental engineering knowledge to complex computer science problems.	Formulation of anatomical landmark spatial geometry and asymmetric angular gaze gating.
CO-2	Design system components and architectures that satisfy specified user needs.	Engineered multi-tier architecture, dual-database layer, and push-to-talk WebM intercom.
CO-3	Conduct investigations of complex problems through experiment and analysis.	Executed 62 automated pytest test fixtures across database, security, and cartographic modules.
CO-4	Create, select, and apply modern engineering and IT tools.	Utilized Python Flask, TensorFlow.js, BlazeFace, PostgreSQL 16, and Git version control.
CO-5	Understand the impact of engineering solutions in societal and ethical contexts.	Implemented privacy-first client-side edge proctoring, eliminating cloud video storage.

A.2 Project Environment Execution Instructions

To execute the verified ExamGuard AI test suite and launch the application locally:

Terminal Command Sequence:

```
# 1. Clone repository and navigate to directory
cd "Online Examination System with Anti-Cheating Detection"

# 2. Activate Python virtual environment
python -m venv venv && source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run automated test suite
python -m pytest tests/ -v
```

5. Launch local application

python app.py